README

🖼️ Implementation of Filters & Edge Detection using OpenCV

This project demonstrates image filtering and edge detection techniques using Python, OpenCV, and Matplotlib in a Jupyter Notebook (`.ipynb`).

The notebook applies different image processing operations such as filtering, grayscale conversion, and edge detection to enhance and analyze images.

📌 Features

- Read and display images using OpenCV
- Convert images from BGR to RGB
- Apply image filters
- Perform grayscale conversion
- Detect edges in images
- Visualize processed images using Matplotlib
- Error handling for missing images

🛠️ Technologies Used

- Python
- OpenCV
- Matplotlib
- NumPy

- Jupyter Notebook

📁 Project Workflow

1. Load the input image
2. Convert image color format
3. Apply image filtering techniques
4. Convert image to grayscale
5. Perform edge detection
6. Display the processed output images

📦 Required Libraries

Install the following libraries before running the notebook:

```
pip install opencv-python matplotlib numpy
```

💻 Sample Python Code

```
import cv2
import matplotlib.pyplot as plt

# Read image
image = cv2.imread('download.jpg')

# Check if image loaded correctly
if image is not None:

    # Convert BGR to RGB
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Convert to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

    # Edge detection
    edges = cv2.Canny(gray, 100, 200)

    # Display edges
    plt.imshow(edges, cmap='gray')
    plt.title("Edge Detection")
    plt.axis("off")
    plt.show()

else:
    print("Error: Image not found or failed to load.")
```

▶️ How to Run

1. Open the Jupyter Notebook
2. Run all cells step by step
3. Ensure the input image exists in the project folder
4. Observe the filtered and edge-detected outputs

⚠️ Common Error

Error

```
error: (-215:Assertion failed) !_src.empty() in function 'cv::cvtColor'
```

Cause

The image was not loaded properly using `cv2.imread()`.

Fix

- Check whether the image path is correct
- Verify the image file exists
- Ensure the image name is correct

✅ Output

The notebook successfully:

- Loads and processes images
- Applies filters
- Detects image edges
- Displays processed outputs using Matplotlib

🌟 Learning Outcomes

Through this project, we learn:

- Basics of image processing
- Image filtering techniques
- Grayscale image conversion
- Edge detection using OpenCV
- Visualization using Matplotlib
- Error handling in image processing

🚀 Future Improvements

- Add advanced filters
- Implement blur and sharpening
- Real-time edge detection using webcam
- Add object detection
- Perform image segmentation

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Contributors 1

Nauxh30

Languages

Jupyter Notebook 100.0%